17

Recursion

Overview

A recursive procedure is a procedure that calls itself. Recursive procedures are important in high-level languages, but the way the system uses the stack to implement these procedures is hidden from the programmer. In assembly language, the programmer must actually program the stack operations, so this provides an opportunity to see how recursion really works.

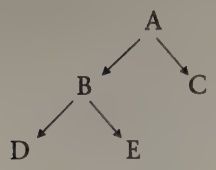
Because you may have had only limited experience with recursion, sections 17.1–17.2 discuss the underlying ideas. Section 17.3 shows how that stack can be used to pass data to a procedure; this topic was also covered in Chapter 14. In sections 17.4–17.5, we apply this method to implement recursive procedures that call themselves once. The chapter ends with a discussion of procedures that make multiple recursive calls.

17.1 The Idea of Recursion

A process is said to be **recursive** if it is defined in terms of itself. For example, consider the following definition of a binary tree:

> A binary tree is either empty, or consists of a single element called the *root,* and whose remaining elements are partitioned into two disjoint subsets (the left and right subtrees), each of which is a binary tree.

Let us apply the definition to show that the following tree T is a binary tree:



Choose A as the root of T. The tree T1, consisting of B, D, and E, is the left subtree of A and the tree T2, consisting of C, is the right subtree. We must show that T1 and T2 are binary trees.

Choose B as the root of T1. The trees $T1_a$ consisting of D and $T1_b$ consisting of E are the left and right subtrees. We must show that $T1_a$ and $T1_b$ are binary trees.

Choose D as the root of $T1_a$. The left and right subtrees of D are empty, and since an empty tree is a binary tree, $T1_a$ is a binary tree. For the same reason, $T1_b$ is a binary tree. Because $T1_a$ and $T1_b$ are binary trees, T1 must be a binary tree.

Now look at T2. It has a root C whose left and right subtrees are empty, so it is a binary tree.

Since T1 and T2 have been shown to be binary trees, tree T must also be a binary tree.

This simple example illustrates the main characteristics of recursive processes:

1. The main problem (showing that T is a binary tree) breaks down to simpler problems (showing that T1 and T2 are binary trees), and each of these problems is solved in exactly the same way as the main problem.
2. There must be an escape case (empty trees are binary trees) that lets the recursion terminate.
3. Once a subproblem has been solved (T1 is shown to be a binary tree), work proceeds on the next step of the original problem (showing that T2 is a binary tree).

17.2 Recursive Procedures

A recursive procedure calls itself. As a first example, consider the factorial of a positive integer. It may be defined nonrecursively as

$$\text{FACTORIAL}(1) = 1$$
$$\text{FACTORIAL}(N) = N \times (N-1) \times (N-2) \times \ldots 2 \times 1 \qquad \text{if } N > 1$$

or, since

$$\text{FACTORIAL}(N-1) = (N-1) \times (N-2) \times \ldots 2 \times 1$$

we may write the following recursive definition:

$$\text{FACTORIAL}(1) = 1$$
$$\text{FACTORIAL}(N) = N \times \text{FACTORIAL}(N-1) \qquad \text{if } N > 1$$

Let's rewrite this as an algorithm for a recursive procedure FACTORIAL:

```
1: PROCEDURE FACTORIAL (input: N, output: RESULT)
2: IF N = 1
3:  THEN
4:    RESULT = 1
5:  ELSE
```

```
6:     call FACTORIAL (input: N - 1, output: RESULT)
7:     RESULT = N x RESULT
8:  END_IF
9:  RETURN
```

In line 7, the value of RESULT on the right side is the value returned by the call to FACTORIAL at line 6.

For $N = 4$, we can trace the actions of the procedure as follows:

```
call FACTORIAL (4,RESULT)  /* begin first call */
call FACTORIAL (3,RESULT)  /* begin second call */
call FACTORIAL (2,RESULT)  /* begin third call */
call FACTORIAL (1,RESULT)  /* begin fourth call */
RESULT = 1
RETURN                     /* end fourth call */
```

The fourth call is the escape case. When it is finished, the third call is resumed at line 7:

```
RESULT = N x RESULT
```

On the right side, $N = 2$ and RESULT is 1, the value computed in the fourth call. We compute

```
RESULT = 2 x 1 = 2
```

and this call ends. The procedure then resumes the second call at line 7. In this call $N = 3$, so we compute

```
RESULT = 3 x RESULT = 3 x 2 = 6
```

which ends this call. Finally the procedure resumes the first call at line 7. In this call $N = 4$, so the result is

```
RESULT = 4 x RESULT = 4 x 6 = 24
```

and this is the value returned by the procedure.

This procedure has the properties of a recursive process that we noticed in the binary tree example. Each call to procedure FACTORIAL works on a simpler version of the original problem (finding the factorial of a smaller number), there is an escape case (the factorial of 1) and once a call has been completed, work continues on the previous call.

As a second example, consider the problem of finding the largest entry in an array A of N integers. If $N = 1$, then the largest entry is the only entry, A[1]. If $N > 1$, the largest entry is either A[N] or the biggest of the entries A[1] . . . A[N – 1]. Here is an algorithm for a procedure.

```
1:  PROCEDURE FIND_MAX(input: N, output: MAX)
2:  IF N = 1
3:   THEN
4:    MAX = A[1]
5:   ELSE
6:    call FIND_MAX(N-1,MAX)
7:    IF A[N] > MAX
8:     THEN
9:      MAX = A[N]
10:    ELSE
11:      MAX = MAX
12:    END_IF
13:  RETURN
```

In lines 7 and 11, the value MAX on the right side is the value returned by the call at line 6.

Let's trace the procedure for an array A of four entries: 10, 50, 20, 4.

```
call FIND_MAX(4,MAX)        /* first call */
call FIND_MAX(3,MAX)        /* second call */
call FIND_MAX(2,MAX)        /* third call */
call FIND_MAX(1,MAX)        /* fourth call */
```

As in the factorial example, the fourth call is the escape case. It returns MAX = A[1] = 10 and exits.

Now the third call resumes at line 7. Because A[2] (= 50) > MAX (= 10), the value returned from this call is MAX = 50.

Next the second call resumes at line 7. Because A[3] (= 20) < MAX (= 50), this call also returns MAX = 50.

Finally, we are back in the first call at line 7. Because A[4] (= 4) < MAX (= 50), this call returns MAX = 50, and this is the value returned to the calling program by the procedure.

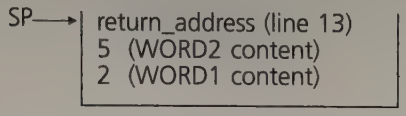
17.3 Passing Parameters on the Stack

As we will see later, recursive procedures are implemented in assembly language by passing parameters on the stack (section 14.5.3). To see how this may be accomplished, consider the following simple program. It places the content of two memory words on the stack, and calls a procedure ADD_WORDS that returns their sum in AX.

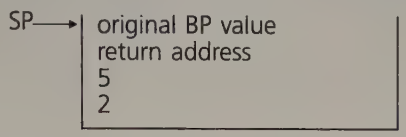
Program Listing PGM17_1.ASM

```
0:   TITLE PGM17_1: ADD WORDS
1:   .MODEL  SMALL
2:   .STACK   100H
3:   .DATA
4:   WORD1    DW     2
5:   WORD2    DW     5
6:   .CODE
7:   MAIN     PROC
8:            MOV    AX,@DATA
9:            MOV    DS,AX
10:           PUSH   WORD1
11:           PUSH   WORD2
12:           CALL   ADD_WORDS
13:           MOV    AH,4CH
14:           INT    21H
15:  MAIN     ENDP
16:  ADD_WORDS       PROC          NEAR
17:  ;adds two memory words
18:  ;stack on entry:   return addr.(top), word2, word1
19:  ;output: AX = sum
20:           PUSH   BP            ;save BP
21:           MOV    BP,SP
22:           MOV    AX,[BP+6]     ;AX gets WORD1
23:           ADD    AX,[BP+4]     ;AX has sum
24:           POP    BP            ;restore BP
25:           RET    4             ;exit
26:  ADD_WORDS       ENDP
27:           END    MAIN
```

After initializing DS, the program pushes the contents of WORD1 and WORD2 on the stack, and calls ADD_WORDS. On entry to the procedure, the stack looks like this:



At lines 20–21, the procedure first saves the original content of BP on the stack, and sets BP to point to the stack top. The result is



Now the data can be accessed by indirect addressing. BP is used for two reasons: (1) when BP is used in indirect addressing, SS is the assumed segment register, and (2) SP itself may not be used in indirect addressing. At line 22, the effective address of the source in the instruction

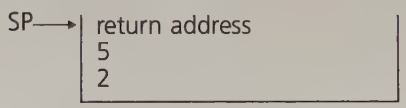
```
MOV AX,[BP+6]
```

is the stack top offset plus 6, which is the location of WORD1 content (2). Similarly, at line 23 the source in

```
ADD AX,[BP+4]
```

is the location of WORD2 content (5).

After restoring BP to its original value at line 24, the stack becomes



To exit the procedure and restore the stack to its original condition, we use

```
RET 4
```

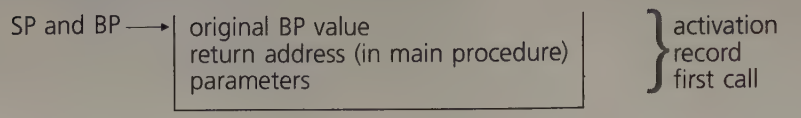
This causes the return address to be popped into IP, and four additional bytes to be removed from the stack.

17.4 *The Activation Record*

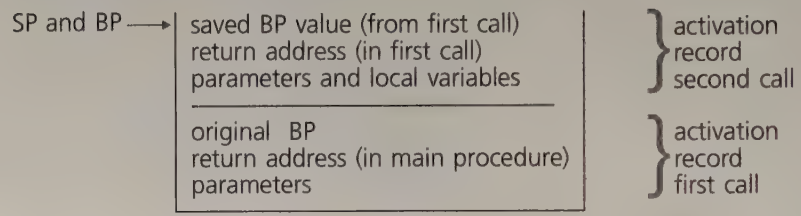
Before attempting to code a recursive procedure, one issue must be resolved. The parameters (and local variables, if any) of the procedure are reinitialized each time the procedure is called. In both examples of section 17.2, the procedure is first called with parameter $N = 4$, then with $N = 3$, then with $N = 2$, then with $N = 1$. When a call has been completed, the procedure resumes the previous call at the point it left off. In order to do so, it must somehow "remember" that point, as well as the values of the parameters and local variables in that call. These values are known as the **activation record** of the call.

To illustrate, suppose we have a procedure that is called once from the main procedure, and then calls itself twice more. Before initiating the first call, the main procedure places the initial activation record on the stack and calls the procedure. The procedure saves BP and sets BP to point to the

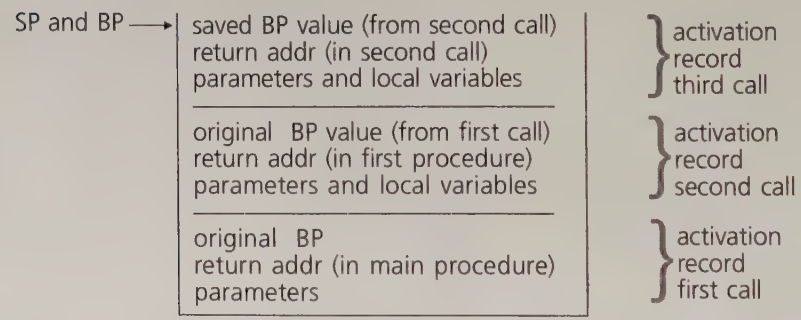
stack top, as was done in the example of the last section. The stack looks like this:



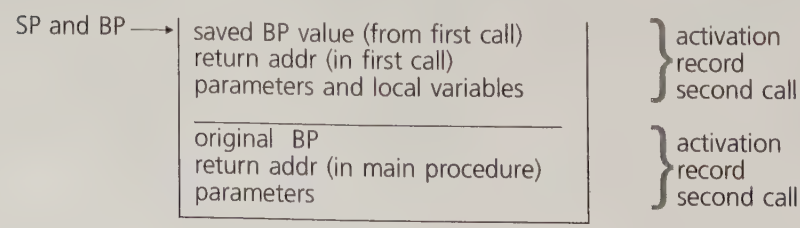
Using BP to access the parameters and local variables, the procedure executes its instructions. Before calling itself, it places the activation record for the next call on the stack. The return address that the recursive call places on the stack is that of the next instruction to be done in the procedure. As the second call begins, the procedure once again saves BP and sets BP to point to the stack top. The result is



Now, as in the first call, the procedure uses BP to access the data for the second call. Before initiating the third call, its activation record is placed on the stack. The third call saves BP and sets it to point to the stack top. The stack becomes

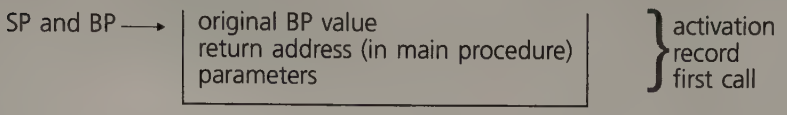


Let's suppose that the third call is the escape case. The result it computes may be placed in a register or memory location so that it is available to the second call when the second call resumes. After the third call is completed, the second call may be resumed by first popping BP to restore its previous value, and executing a return. The return places in IP the address of the next instruction to be done in the second call. As part of the return, the third call's parameters and local variables are popped off the stack and discarded, as was done in the example in the last section. The stack becomes



Now the second call resumes. It picks up the result of the third call and executes to completion. When it has finished and stored the result, the

stack is once again popped into BP, and control returns to the first call. As before, the second call's data are discarded. Now the stack looks like this:



When the first call is done, the procedure restores BP to its original value, and control passes to the main procedure. As before, the parameters are discarded. The procedure stores the final result in a place where the main procedure can pick it up.

17.5 Implementation of Recursive Procedures

In this section, we show how recursive procedures may be implemented in assembly language.

Example 17.1 Code the FACTORIAL procedure of section 17.2. Call it in a program to compute the factorial of 3.

Solution: To make the code easier to follow, the algorithm is repeated here:

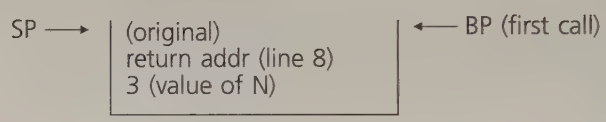
```
1: PROCEDURE FACTORIAL (input: N, output: RESULT)
2: IF N = 1
3:  THEN
4:   RESULT = 1
5:  ELSE
6:   call FACTORIAL (input: N - 1, output: RESULT)
7:   RESULT = N x RESULT
8: END_IF
9: RETURN
```

Program Listing PGM17_2.ASM

```
0:  TITLE PGM17_2: FACTORIAL PROGRAM
1:  .MODEL  SMALL
2:  .STACK  100H
3:  .CODE
4:  MAIN   PROC
5:      MOV   AX,3              ;N = 3
6:      PUSH  AX                ;N on stack
7:      CALL  FACTORIAL         ;AX has 3 factorial
8:      MOV   AH,4CH
9:      INT   21H               ;dos return
10: MAIN   ENDP
11: FACTORIAL      PROC         NEAR
12: ; computes N factorial
13: ; input:stack on entry - ret. addr.(top), N
14: ; outputAX
15:     PUSH  BP                ;save BP
16:     MOV   BP,SP             ;BP pts to stacktop
17: ;if
18:     CMP   WORD PTR[BP+4],1 ;N = 1?
```

```
19:       JG    END_IF            ;no, N1
20: ;then
21:       MOV   AX,1              ;result = 1
22:       JMP   RETURN            ;go to return
23: END_IF:
24:       MOV   CX,[BP+4]         ;get N
25:       DEC   CX                ;N-1
26:       PUSH  CX                ;save on stack
27:       CALL  FACTORIAL         ;recursive call
28:       MUL   WORD PTR[BP+4]    ;RESULT = N*RESULT
29: RETURN:
30:       POP   BP                ;restore BP
31:       RET   2                 ;return and discard N
32: FACTORIAL  ENDP
33:       END   MAIN
```

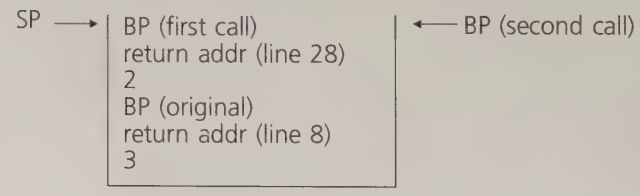
The testing program puts 3 on the stack and calls FACTORIAL. At lines 15 and 16 the procedure saves BP and sets BP to point to the stack top. The stack looks like this:



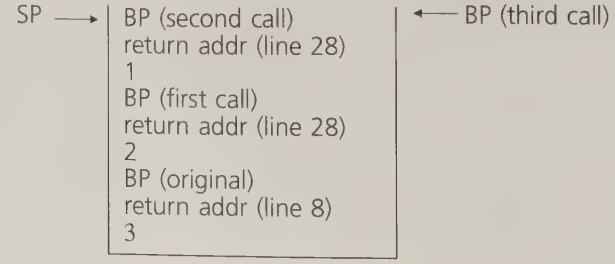
Now, at line 18 the current value of *N* is examined. We must use CMP WORD PTR [BP+4],1 rather than CMP [BP+4],1 because the assembler cannot tell from the source operand 1 whether to code this as a byte or word instruction.

Because $N \neq 1$, in lines 24–26 the data for the next call are prepared by retrieving the current value of N, decrementing it, and saving it on the stack.

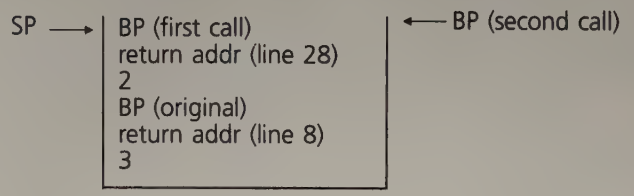
At line 27, the second call ($N = 2$) is made. Once again, at lines 15 and 16, BP is saved and BP is set to point to the stack top. The stack becomes



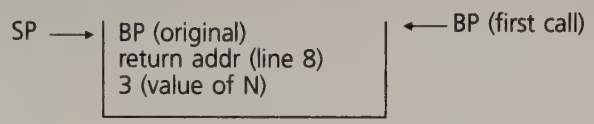
Since *N* is still not 1, the procedure calls itself one more time, and the stack looks like this:



Since *N* is now 1, the recursion can terminate. At line 21, the procedure places RESULT = 1 in AX, restores BP to its value in the second call and returns. The RET 2 at line 31 causes the return address in the second call (line 28 in the listing) to be placed in IP. RET 2 also causes parameter 1 to be popped off the stack. The stack becomes



Now execution of the second call continues at line 28. Because the result of the third call is in AX, the procedure can multiply it by the current value of *N*, yielding RESULT = 2 × 1 = 2. The new result remains in AX. With this call complete, BP is restored and the first call resumed at line 28. The stack is now



As before, the latest result is multiplied by *N*, yielding RESULT = 3 × 2 = 6. Control passes to line 8 in the main program, with the value of the factorial in AX.

Example 17.2 Code procedure FINDMAX of section 17.2, and test it in a program.

Solution: The algorithm for the procedure is reproduced here:

```
1: PROCEDURE FIND_MAX (input: N, output: MAX)
2: IF N = 1
3:  THEN
4:   MAX = 1
5:  ELSE
6:   call FIND_MAX(N - 1,MAX)
7:   IF A[N] > MAX
8:    THEN
9:     MAX = A[N]
10:   ELSE
11:    MAX = MAX /* value returned by call at line 6 */
12:  ENDIF
13: RETURN
```

Program Listing PGM17_3.ASM

```
0:  TITLE PGM17_3: FIND_MAX
1:  .MODEL  SMALL
2:  .STACK  100H
3:  .DATA
4:  A           DW     10,50,20,4
5:  .CODE
6:  MAIN     PROC
7:           MOV    AX,@DATA
8:           MOV    DS,AX          ;initialize DS
9:           MOV    AX,4           ;no. of elts in array
10:          PUSH   AX             ;parameter on stack
11:          CALL   FIND_MAX       ;returns MAX in AX
12:          MOV    AH,4CH
13:          INT    21H            ;dos exit
14: MAIN     ENDP
15: FIND_MAX    PROC    NEAR
16: ; finds the largest element in array A of N elements
17: ; input: stack on entry - ret. addr. (top), N
18: ; output: AX largest element
19:          PUSH   BP             ;save BP
20:          MOV    BP,SP          ;BP pts to stacktop
21: ;if
22:          CMP    WORD PTR [BP+4],1  ;N=1?
23:          JG     ELSE_          ;no, go to set up next call
24: ;then
25:          MOV    AX,A           ;MAX = A[1]
26:          JMP    END_IF1
27: ELSE_:
28:          MOV    CX,[BP+4]      ;get N
29:          DEC    CX             ;N-1
30:          PUSH   CX             ;save on stack
31:          CALL   FIND_MAX       ;returns MAX in AX
32: ;if
33:          MOV    BX,[BP+4]      ;get N
34:          SHL    BX,1           ;2N
35:          SUB    BX,2           ;2(N-1)
36:          CMP    A[BX],AX       ;A[N] > MAX?
37:          JLE    END_IF1        ;no, go to return
38: ;then
39:          MOV    AX,A[BX]       ;yes, set MAX = A[N]
40: END_IF1:
41:          POP    BP             ;restore BP
42:          RET    2              ;return and discard N
43: FIND_MAX       ENDP
44:          END    MAIN
```

The stacking of the activation records during the recursive calls in this example is similar to that of example 17.1, and is not shown here (see exercises).

At line 32, the procedure begins preparation for comparison of A[N] with the current value of MAX in AX. Recall from chapter 10 that the offset location of the *N*th element of a word array A is A + 2 × (N – 1). Lines 33-35 put 2 × (N – 1) in BX, so that based mode may be used in the comparison at line 36. If MAX > A[*N*], we can leave it in AX, which means that the ELSE statement at line 11 of the algorithm need not be coded.

17.6 More Complex Recursion

In the preceding examples, the code for recursive procedures has involved only one recursive call; for example, the only call that procedure FACTORIAL (*N*) makes is to FACTORIAL (*N* – 1). However, it is possible that the code for a recursive procedure may involve multiple recursive calls.

As an example, suppose we would like to write a procedure to compute the binomial coefficients C(*n*, *k*). These are the coefficients that appear in the expansion of $(x + y)^n$. The expansion takes the form

$$(x+y)^n = C(n,0)x^n y^0 + C(n,1)x^{n-1}y^1 + C(n,2)x^{n-2}y^2 + \ldots + C(n,n-1)x^1 y^{n-1} + C(n,n)x^0 y^n$$

These coefficients also are used in the construction of Pascal's Triangle. For $n = 4$, the triangle is

C(0, 0)
C(1, 0) C(1, 1)
C(2, 0) C(2, 1) C(2, 2)
C(3, 0) C(3, 1) C(3, 2) C(3, 3)
C(4, 0) C(4, 1) C(4, 2) C(4, 3) C(4, 4)

The coefficients satisfy the following relation:

$$C(n, n) = C(n, 0) = 1$$
$$C(n, k) = C(n-1, k) + C(n-1, k-1) \quad \text{if} \quad n > k > 0$$

This means that in the triangle, the entries along the edges are all 1's, and an interior entry is the sum of the entries in the row above immediately to the left and right. So the triangle computes to

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1

Let's apply the preceding definition to compute *C*(3, 2):

$C(3, 2) = C(2, 2) + C(2, 1)$
$C(2, 2) = 1$
$C(2, 1) = C(1, 1) + C(1, 0)$
$C(1, 1) = 1$
$C(1, 0) = 1$
So $C(2, 1) = 1 + 1 = 2$
and $C(3, 2) = 1 + 2 = 3$

Here is an algorithm for a procedure to compute C(n, k):

```
PROCEDURE BINOMIAL (input: N, K; output: RESULT)
IF (K = N) OR (K = 0)
 THEN
  RESULT = 1
 ELSE
  CALL BINOMIAL(N-1,K,RESULT1)
  CALL BINOMIAL(N-1,K-1,RESULT2)
  RESULT = RESULT1 + RESULT2
RETURN
```

Example 17.3 Code the BINOMIAL procedure and call it in a program to compute $C(3, 2)$.

Solution:

Program Listing PGM17_4.ASM

```
0:  TITLE PGM17_4: BINOMIAL COEFFICIENTS
1:  .MODEL  SMALL
2:  .STACK  100H
3:  .CODE
4:  MAIN     PROC
5:           MOV   AX,2          ;K=2
6:           PUSH  AX
7:           MOV   AX,3          ;N=3
8:           PUSH  AX
9:           CALL  BINOMIAL      ;AX = RESULT
10:          MOV   AH,4CH
11:          INT   21H           ;DOS EXIT
12: MAIN     ENDP
13: BINOMIAL    PROC     NEAR
14:          PUSH  BP
15:          MOV   BP,SP
16:          MOV   AX,[BP+6]     ;get K
17: ;if
18:          CMP   AX,[BP+4]     ;K=N?
19:          JE    THEN          ;yes, nonrecursive case
20:          CMP   AX,0          ;K=0?
21:          JNE   ELSE_         ;no, recursive case
22: THEN:
23:          MOV   AX,1          ;RESULT = 1
24:          JMP   RETURN
25: ELSE_:
26: ;compute C(N-1,K)
27:          PUSH  [BP+6]        ;save K
28:          MOV   CX,[BP+4]     ;get N
29:          DEC   CX            ;N-1
30:          PUSH  CX            ;save N-1
31:          CALL  BINOMIAL      ;RESULT1 in AX
32:          PUSH  AX            ;save RESULT1
33: ;compute C(N-1,K-1)
34:          MOV   CX,[BP+6]     ;get K
35:          DEC   CX            ;K-1
36:          PUSH  CX            ;save K-1
37:          MOV   CX,[BP+4]     ;get N
38:          DEC   CX            ;N-1
39:          PUSH  CX            ;save N-1
40:          CALL  BINOMIAL      ;RESULT2 in AX
41: ;compute C(N,K)
42:          POP   BX            ;get RESULT1
43:          ADD   AX,BX         ;RESULT = RESULT1 + RESULT2
44: RETURN:
45:          POP   BP            ;restore BP
46:          RET   4             ;return and discard N and K
47: BINOMIAL      ENDP
48:          END   MAIN
```

Procedure BINOMIAL differs from the procedures of examples 17.1 and 17.2 in the following ways:

1. There are two escape cases, k = n or k = 0; in both cases, the call returns 1 in AX (line 23).
2. In the general case, computation of $C(n, k)$ involves two recursive calls, to compute $C(n - 1, k)$ and $C(n - 1, k - 1)$.

All calls to BINOMIAL return the result in AX. After $C(n - 1, k)$ is computed (line 31), the result (RESULT1 in the algorithm) is pushed onto the stack (line 32). At line 40, $C(n - 1, k - 1)$ is computed and the result (RESULT2 in the algorithm) will be in AX. At lines 42–43, RESULT1 is popped into BX and added to RESULT2, so that AX will contain $C(n, k) = C(n - 1, k) + C(n - 1, k - 1)$.

To completely understand how procedure BINOMIAL works, you are encouraged to trace the effect of the procedure on the stack, as was done in example 17.1.

Summary

- Recursive problem solving has the following characteristics: (1) The main problem breaks down to simpler problems, each of which is solved in the same way as the main problem; (2) there is a nonrecursive escape case; and (3) once a subproblem has been solved, work proceeds to the next step of the original problem.
- In assembly language, recursive procedures are implemented as follows: The calling program places the activation record for the first call on the stack and calls the procedure. The procedure uses BP to access the data it needs from the stack. Before initiating a recursive call, a procedure places the activation record for the call on the stack and calls itself. When a call is completed, BP is restored, the return address popped into IP, and the data for the completed call popped off the stack.
- The code for a procedure may involve more than one recursive call. Intermediate results may be saved on the stack, and retrieved when the original call resumes.

Glossary

activation record	Values of the parameters, local variables, and return address of a procedure call
recursive process	A process that is defined in terms of itself

Exercises

1. Write a recursive definition of a^n, where n is a nonnegative integer.
2. Ackermann's function is defined as follows for nonnegative integers m and n:

$$A(0, n) = n + 1$$

$$A(m, 0) = A(m - 1, 1)$$

$$A(m, n) = A(m - 1, A(m, n - 1)) \quad \text{if } m, n \neq 0$$

   Use the definition to show that A(2,2) = 7.
3. Trace the steps in example 17.2 (PGM17_3.ASM) and show the stack
   a. At line 20 in the initial (first) call to FIND_MAX.
   b. At line 20 in the second call to FIND_MAX.
   c. At line 20 in the third call to FIND_MAX.
   d. At line 20 in the fourth call to FIND_MAX. This is the escape case.
   e. At line 42 in the completion of the third call to FIND_MAX (after RET 2 has been executed). Also give the contents of AX.
   f. At line 42 in the completion of the second call to FIND_MAX (after RET 2 has been executed). Also give the contents of AX.
   g. At line 42 in the completion of the first call to FIND_MAX (after RET 2 has been executed). Also give the contents of AX. This is the value returned by the procedure.

Programming Exercises

4. Write a recursive assembly language procedure to compute the sum of the elements of a word array. Write a program to test your procedure on a four-element array.
5. The Fibonacci sequence 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, . . . may be defined recursively as follows:

$$F(0) = F(1) = 1$$

$$F(n) = F(n - 1) + F(n - 2) \quad \text{if } n > 1$$

   Write a recursive assembly language procedure to compute $F(n)$, and call it in a testing program to compute $F(7)$.